

International Conference on Machine Learning and Data Engineering

Cost-Efficient Blockchain Architecture for Agricultural Traceability: Transaction-Throughput Evaluation Using IPFS, Validity Rollups, and ERC-1167 Smart Contracts

Raviprakash B^a, Shabari^b, Riyaz^{b,*}, Rama Moorthy H^b, Shyam R^a, Amritakala Shetty^b^aDepartment of Computer Science and Information Technology, Jain (Deemed to be University), Bangalore 560069, Karnataka, India^bDepartment of Computer Applications, Nitte (Deemed to be University), Mangaluru, 575007, Karnataka, India

Abstract

Agricultural traceability requires verifiable storage of certificates, images, videos, custody records, and frequent multi-party transactions without excessive replicated state or contract cost. This paper integrates encrypted InterPlanetary File System (IPFS) storage, compact Ethereum commitments, validity-rollup execution, and ERC-1167 minimal-proxy contracts. A corrected Ethereum Virtual Machine model shows that writing a 10 MB object into new contract state requires at least 6.554×10^9 storage gas. A representative three-slot content identifier requires approximately 6.0×10^4 storage gas, a 99.9991% reduction before shared overhead. Five application-side batches containing 25–200 transactions give mean processing throughput of 325.96 transactions/s, a 2.28% coefficient of variation, and an elapsed-time fit of $R^2 = 0.9992$. The optimized prototype reduces the principal deployment field from 1,732,242 to 1,222,091 gas (29.45%). Aggregate gas across six workflow functions falls from 573,225 to 457,071 gas (20.26%). The reported values and original experimental figures are retained without alteration. The paper separates storage, application-pipeline, and contract-execution boundaries and specifies formal invariants, persistence, authorization, clone safety, and reproducible validation.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: Agricultural supply chain; Blockchain; Ethereum; IPFS; Validity rollup; ERC-1167; Gas optimization; Traceability

1. Introduction

Agricultural supply chains cross organizational and geographic boundaries. A product lot may be created by a farmer, inspected by a cooperative, consolidated by a distributor, transported under monitored conditions, divided by a retailer, and queried by a consumer. Conventional databases execute this workflow efficiently but place alteration and availability control in one organization. Public blockchains provide shared ordering and tamper-evident execution, yet replicating every image, certificate, and sensor stream across validating nodes is economically unsuitable.

* Riyaz

E-mail address: riyaz.konaje@nitte.edu.in

1877-0509 © 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Recent reviews report that blockchain can strengthen provenance, traceability, transparency, auditability, and stakeholder trust in agri-food and broader supply chains. They also identify recurring barriers involving scalability, interoperability, privacy, data reliability, governance, and integration with physical supply-chain processes [1–4]. Adoption-oriented reviews further show that deployment conditions vary across applications and countries and depend on enabling blockchain characteristics, food-chain integration, institutional settings, regulation, and infrastructure readiness [5–7].

The work contributes a layered IPFS–Ethereum architecture with rollup execution and corrected slot-level storage equations. It also provides consistent throughput and gas reanalysis, a versioned ERC-1167 clone model, and formal security and reproducibility conditions. Gas values are reported as gas rather than Gwei, and application processing is not presented as global Ethereum throughput.

2. Related Work and Technical Basis

Fiore and Mongiello provide a comprehensive review of the advantages and drawbacks of blockchain in agri-food supply chains [2]. Lv et al. systematically review blockchain-based agricultural-product traceability research and its technical and application characteristics [3]. Ellahi et al. examine blockchain-based food-traceability frameworks, their integration with emerging technologies, and areas requiring improvement [8]. Khor et al. propose a scalable lightweight public-blockchain protocol for interoperable ownership transfer of tagged supply-chain goods [9]. Risso et al. develop methodological guidelines for constructing hybrid-simulation models of blockchain-supported supply-chain traceability systems [10].

IPFS-based systems can keep bulk objects outside blockchain state while retaining content-addressed references. Sangeeta and Nam combine blockchain and IPFS with keyword-searchable metadata in a vehicular-network storage system [11]. Dwivedi et al. propose an Ethereum-smart-contract and IPFS scheme that includes distributed storage, device authentication, and data-access policies for fog and industrial-IoT environments [12]. Lamichhane and Herbke introduce verifiable decentralized IPFS clusters to address the trust and single-provider risks of conventional pinning services [13]. These studies motivate content-addressed off-chain storage, but production use still requires explicit encryption, replicated pinning, retrieval verification, and key governance.

Kostamis et al. compare Ethereum dApp data-management methods in terms of storage cost and retrieval performance, supporting an explicit treatment of storage representation rather than an undifferentiated cost-per-byte assumption [14]. Lavour et al. propose an on-demand modular zk-rollup design for application-specific transactions [15], while Chaliasos et al. decompose ZK-rollup costs and benchmark major ZK-EVM implementations, including zkSync Era and Polygon zkEVM [16]. Grandjean et al. measure participation and decentralization in Ethereum’s proof-of-stake consensus layer; this layer-1 validator function is distinct from application-level rollup operation [17]. Ebrahimi et al. provide a large-scale empirical study of proxy-pattern use in Ethereum dApps, motivating explicit treatment of proxy provenance and state separation [18]. At the optimization level, Li et al. detect recurring gas-expensive smart-contract patterns [19], whereas Liu and Song reduce invocation gas by reordering bytecode-level function dispatch [20]. A recent AIDE study investigates scalable blockchain flash loans for optimizing arbitrage opportunities in automated market makers, emphasizing mathematical analysis, heuristic decision-making, throughput, and risk mitigation in decentralized finance [21].

3. System Model and Hybrid Architecture

3.1. Actors and Requirements

The actors are farmers, inspectors, logistics providers, retailers, customers, a dApp service, IPFS providers, a rollup operator, and Ethereum validators. Farmers and inspectors generate evidence; logistics actors provide a custody update; retailers keep lots and orders; and customers confirm permitted provenance. The ledger does not ensure the truth of the submitted statements, only ordering and authorization. Device attestation, accredited inspection, sampling and dispute governance are still required.

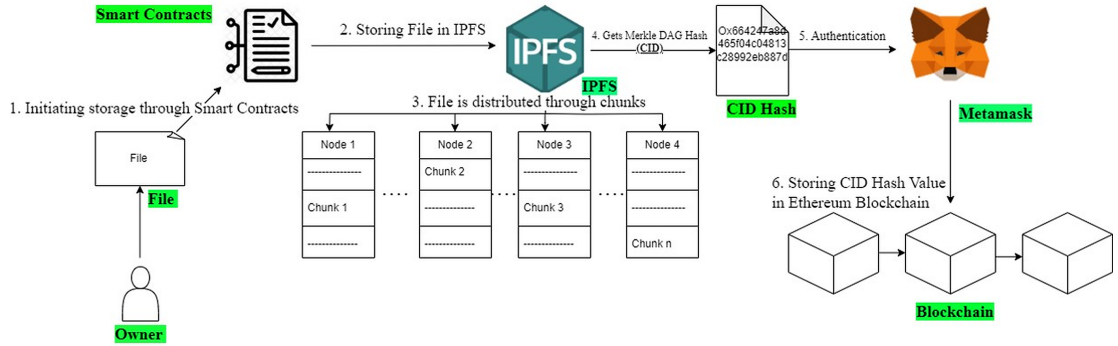


Fig. 1. Original proposed Ethereum–IPFS architecture.

Each lot stores a unique identifier, custodian, lifecycle state, quantity, version, and evidence commitment. Required properties are tamper evidence, bounded contract work, encrypted evidence, tolerance of provider failure, explicit rollup settlement status, replay protection, and append-only corrections.

3.2. Evidence and Workflow Layers

Figure 1 retains the original Ethereum–IPFS architecture. The client canonicalizes, signs, and encrypts evidence; IPFS stores ciphertext under independently pinned CIDs; a validity-rollup contract validates roles, versions, quantities, and commitments; and Ethereum provides settlement.

For lot j , the authoritative record is

$$R_j = (id_j, owner_j, state_j, q_j, c_j, h_j, t_j, v_j), \quad (1)$$

where q_j is quantity, c_j is a compact CID, h_j is a canonical metadata hash, t_j is transition time, and v_j is a monotonic version. Long histories are reconstructed from events rather than unbounded state arrays.

Algorithm 1 Evidence publication and state transition

Require: Lot L , files F , event e , actor key sk

- 1: Validate role, state, quantity, nonce, and version
 - 2: $P \leftarrow \text{CANONICALPACKAGE}(L, F, e); \sigma \leftarrow \text{SIGN}(sk, H(P))$
 - 3: $(K, C) \leftarrow \text{ENCRYPT}(P||\sigma); cid \leftarrow \text{IPFSADD}(C)$
 - 4: Recompute cid and obtain independent pin acknowledgements
 - 5: Submit $(L, e, \text{COMPACT}(cid), H(P), v_L)$ to the rollup
 - 6: Expose inclusion and Ethereum-settlement status separately
-

The algorithm first checks the role, lifecycle state, quantity, nonce, record version of the actor to avoid unauthorized or outdated updates. It then canonicalizes and digitally signs the evidence package to offer deterministic integrity and actor accountability. The signed package is encrypted and then uploaded to IPFS, and then the CID of that package is recomputed and then is pinned independently for verifiable availability. A compact CID, metadata hash, event and current version are sent to the validity rollup for batched verification. Lastly, the application provides for reporting of the rollups inclusion without the finality of Ethereum, there by ensuring that the preliminary acceptance is not considered finality.

The workflow in Fig. 2 remains the same as the original order to delivery workflow. It starts with evidence enquiry, progresses to reservation and payment allocation, and ends at recorded delivery. Media is stored in IPFS, contracts store contract state and payment privileges. All transitions provide the anticipated version to avoid stale concurrent updates.

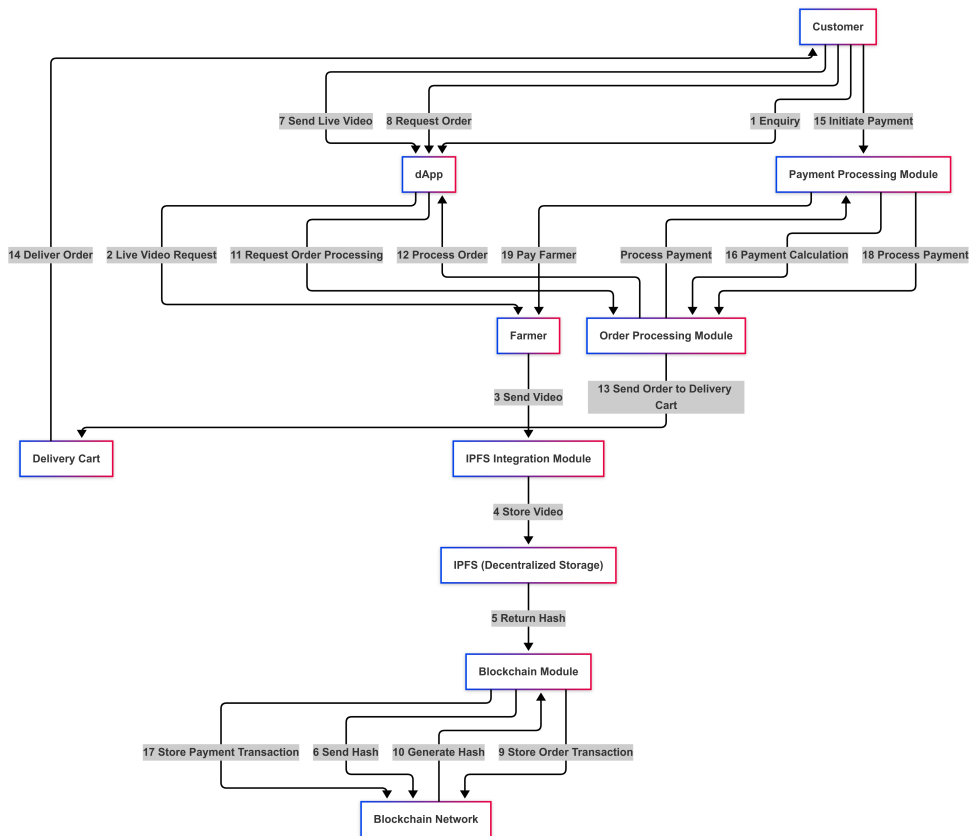


Fig. 2. Original agricultural order-to-delivery system design.

3.3. Lifecycle Protocol

The baseline lifecycle is

Created → Listed → Reserved → InTransit → Delivered. (2)

Cancellation is only allowed in policy-defined nonterminal states. If there is a dispute, commercial settlement is frozen without deletion of the custody record. Resolution adds a signed event giving a reference to the disputed version. The number of children in a lot split must be bounded and accounting for losses must be declared; a merge can only be accepted if the attributes (commodity, unit, policy) are compatible. Payment processing records entitlements and follows a pull with withdrawal approach, meaning that if one fails to receive its payments, no other party can prevent the completion of the payments.

Each signed request contains a chain identifier, the address of the contract, the action type, the lot/order identifier, the expected version, the nonce and the expiry. This domain separation is used to make sure that a valid signature cannot be replayed in a different deployment. The confirmation states that a client is presented with are application/rollup acceptance, rollup commitment or proof progress, and Ethereum settlement. The interface should not imply that it represents an Ethereum settlement.

4. Storage and Rollup Models

4.1. Hybrid Storage Cost

Let F be the size of evidence in bytes and $w = 32$ be the size of EVM word. A lower bound of writing an all-new payload to state is

$$N_s = \left\lceil \frac{F}{w} \right\rceil, \quad G_{state} \geq N_s G_{set}, \quad (3)$$

where $G_{set} = 20,000$ gas denotes a zero-to-nonzero slot term in the stated model [14]. For all-nonzero calldata,

$$G_{calldata} \approx 16F. \quad (4)$$

For a hybrid record using k new slots,

$$G_{hybrid} \approx kG_{set} + G_{logic} + G_{tx}. \quad (5)$$

For $F = 10 \times 1024^2 = 10,485,760$ bytes, $N_s = 327,680$ and $G_{state} \geq 6,553,600,000$ gas. All-nonzero calldata contributes approximately 167,772,160 gas. A representative three-slot commitment contributes approximately 60,000 storage gas, yielding

$$S = 1 - \frac{60,000}{6,553,600,000} = 99.999084\%. \quad (6)$$

The effective price for ETH is p_g (in ETH/price), and the price in ETH/gas is $C_{ETH} = G_{used} p_g$. The charge for persistence, key management, and providers are distinct costs of IPFS.

4.2. Validity-Rollup Execution

Let G_f be fixed layer-1 submission cost, G_v proof-verification cost, $G_d(n)$ data-publication cost and $G_{L2,i}$ be the cost of the gas used for i transition. Amortized cost is

$$\bar{G}(n) = \frac{G_f + G_v + G_d(n)}{n} + \frac{1}{n} \sum_{i=1}^n G_{L2,i}. \quad (7)$$

The higher batch sizes spread out the fixed terms, but can add latency. If the actor-role relation holds, if the nonce is unused, and if the supplied version is current, the transition $x_{i+1} = f(x_i, u_i)$ is accepted.

5. Application-Side Performance Results

The retained study contains batches of 25, 50, 100, 150, and 200 transactions with elapsed times 0.0755, 0.1555, 0.2973, 0.4660, and 0.6272 s. Raw transaction hashes were not available, or there were no repetitions, hardware

metadata, or there was no exact timing boundary. The observations are not measured by the block time in Sepolia or the TPS per network, but rather the application-pipeline. The throughput is re-calculated as $Q_i = N_i/T_i$.

Table 1. Reanalysed application-side batch measurements.

Transactions N	Time T (s)	N/T (tx/s)
25	0.0755	331.13
50	0.1555	321.54
100	0.2973	336.36
150	0.4660	321.89
200	0.6272	318.88

In Table 1 we display the main application side performance evidence for the proposed architecture for a number of batches ranging from 25 to 200 transactions. An 8-fold increase in the batch size does not affect the throughput as it varies between 318.88 and 336.36 tx/s over the evaluated range, indicating consistent processing. This result is shown to be scalable for the prototype pipeline without being dependent on the throughput of the Ethereum network or settlement.

Ordinary least squares gives

$$\widehat{T} = 0.003145N - 0.005938, \quad (8)$$

with $R^2 = 0.999165$ and RMSE 0.00582 s. Mean throughput is 325.96 tx/s; sample standard deviation is 7.43 tx/s; and

$$CV = \frac{7.43}{325.96} \times 100 = 2.28\%. \quad (9)$$

Low dispersion does not mean saturation or settlement throughput but stability in processing, for the range measured.

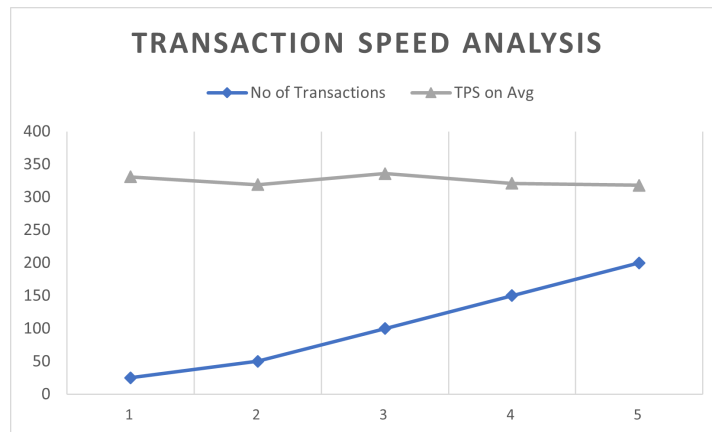


Fig. 3. Original transaction-speed analysis graph.

Figure 3 shows that processing time increases almost linearly as the batch size grows from 25 to 200 transactions. The closely grouped throughput values indicate stable application-side processing across the measured batches. These results describe the prototype processing pipeline and should not be interpreted as Ethereum network or settlement throughput.

6. Minimal-Proxy Contract Model

Figure 4 presents the ERC-1167 deployment model used in this study. An actor interacts with a factory that creates independently initialized proxy instances, represented by state sites S1–S3. Each proxy retains its own workflow state while forwarding function execution through a delegate call to one locked reference implementation. Existing proxies preserve a fixed implementation target, whereas incompatible logic is introduced through a separately registered implementation version.

If D is the gas required to deploy one complete standalone contract, deploying n independent contracts costs

$$C_{full}(n) = nD. \quad (10)$$

In the architecture shown in Fig. 4, let I be the one-time implementation deployment gas, F_c the factory and registry allocation, P the per-proxy creation gas, and A_i the initialization gas of proxy i . The corresponding clone-based deployment cost is

$$C_{clone}(n) = I + F_c + \sum_{i=1}^n (P + A_i). \quad (11)$$

If initialization cost is approximately constant, $A_i = A$, this becomes $C_{clone}(n) = I + F_c + n(P + A)$. When $D > P + A$, clone deployment becomes economical for

$$n > \frac{I + F_c}{D - (P + A)}. \quad (12)$$

This threshold is for deployment savings only. Delegation overhead does not cancel out when delegating runtime lifecycle calls and these calls need to be considered separately from costs of creating factories and proxies.

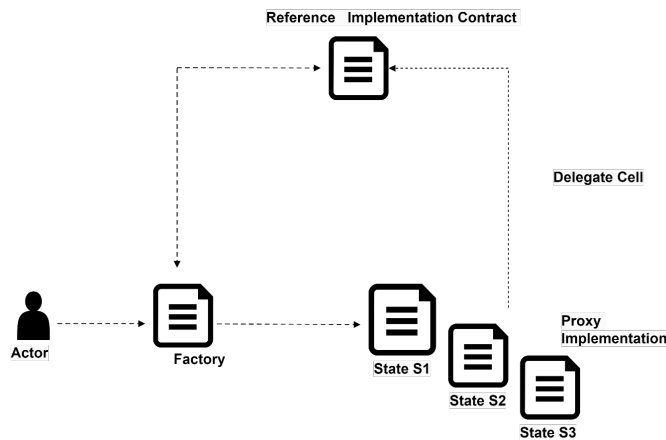


Fig. 4. Original EIP-1167 minimal-proxy architecture.

Thus, the figure corresponds directly to the model, where the $I + F_c$ term is the shared reference implementation and factory, and each state site adds its own $P + A_i$ term. To prevent clone-takeover or provenance risks being introduced by the deployment savings, atomic initialization and implementation-address verification is required.

7. Smart-Contract Gas Results

The gas, transaction-cost, and execution-cost fields are reported on the retained artifacts. The first “Gwei” label, which they used, was dimensionally wrong: cost of money is gas x price. No linked source hashes, repeated trials – results come from traditional and optimized prototype variants, not just ERC-1167.

Table 2. Prototype deployment measurements.

Metric (gas)	Traditional	Optimized	Reduction
Gas field	1,732,242	1,222,091	29.45%
Transaction cost	1,506,297	1,062,687	29.45%
Execution cost	1,350,817	938,997	30.49%

Table 2 quantifies the deployment-level benefit of the optimized prototype. The reported gas and transaction-cost fields decrease from 1,732,242 to 1,222,091 gas and from 1,506,297 to 1,062,687 gas, respectively, corresponding to 29.45% reductions. Execution cost shows the largest improvement, falling by 30.49%, which confirms a consistent reduction across all reported deployment metrics.

Table 3. Supply-chain function gas measurements.

Function	Original	Optimized	Reduction
createShipment	160,308	129,097	19.47%
startShipment	51,137	45,632	10.77%
completeShipment	120,107	101,050	15.87%
getShipment	25,319	19,973	21.11%
getShipmentCount	15,121	10,130	33.01%
getAllShipments	201,233	151,189	24.87%
Total	573,225	457,071	20.26%

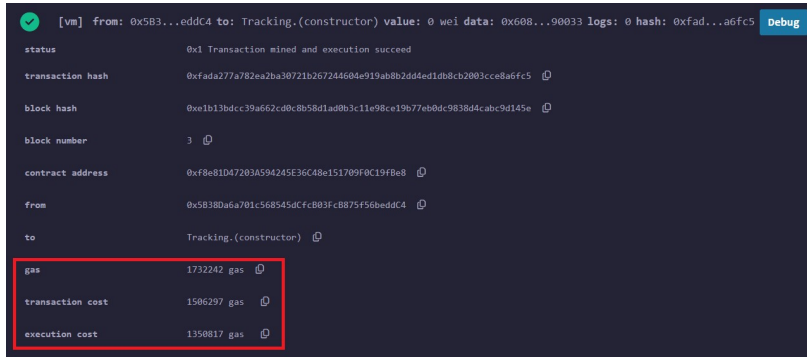
Table 3 demonstrates that every measured supply-chain function requires less gas after optimization, reducing the aggregate value from 573,225 to 457,071 gas (20.26%). The largest improvement occurs for getShipmentCount at 33.01%, followed by getAllShipments at 24.87%, while startShipment records the smallest reduction at 10.77%. The results show that the optimization benefits both state-changing operations and retrieval-oriented functions.

For workload frequencies f_i ,

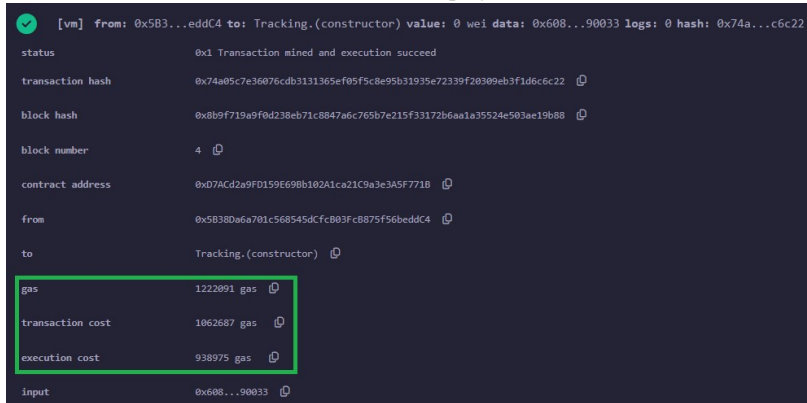
$$R_{workload} = 100 \left(1 - \frac{\sum_i f_i G_{opt,i}}{\sum_i f_i G_{orig,i}} \right). \quad (13)$$

In Eq. 13, f_i is the expected invocation frequency of function i , while $G_{orig,i}$ and $G_{opt,i}$ are its original and optimized gas values from Table 3. The denominator represents the gas required by the original contract for a given workload mix, and the numerator represents the corresponding optimized cost; hence, $R_{workload}$ gives the percentage reduction for that same workload. It is assumed that no production frequency distribution exists, and the reported 20.26% is the average of various functions measured. An off-chain eth_call does not cost any gas on the user’s transaction, however it is not ideal for unbounded getters for the same reasons as getters on nodes and RPCs.

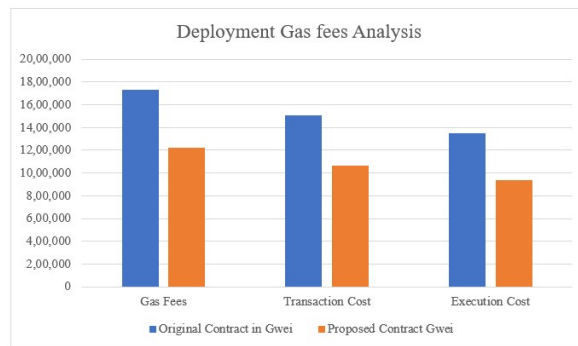
The 99.9991% storage value and 325.96 tx/s processing value describe different boundaries. The 29.45% deployment and 20.26% function-gas reductions describe a third boundary. All values are retained as originally derived and are not combined into one global performance claim.



(a) Traditional deployment



(b) Optimized deployment



(c) Comparative analysis

Fig. 5. Deployment evidence: complete constructor transaction receipts from Pictures 8 and 9 and the original comparative gas analysis. Values are interpreted as gas units.

Figure 5 presents the complete mined constructor receipts for the traditional and optimized tracking contracts. Picture 8 records 1,732,242 gas, a transaction cost of 1,506,297 gas, and an execution cost of 1,350,817 gas; Picture 9 records 1,222,091, 1,062,687, and 938,975 gas, respectively. These receipts support the reduction shown in Table 2 and panel (c), although the optimized execution-cost entry in the table differs from Picture 9 by 22 gas and should be verified before final publication.

8. Conclusion and Future Work

This paper presented an agricultural traceability framework using encrypted IPFS evidence, Ethereum commitments, validity-rollup execution, and versioned ERC-1167 clones. The original 10 MB model yields a 99.9991% representative state-write reduction. Retained batches produce 325.96 tx/s mean processing throughput, 2.28% vari-

ation, and $R^2 = 0.9992$. The optimized prototype reduces the principal deployment gas field by 29.45%. Aggregate workflow-function gas decreases by 20.26%. The original evidence, figures, tables, and result values are preserved while the analysis clarifies their distinct boundaries and limitations.

Future work will implement the full factory, clone, evidence, and workflow interfaces on a specified Ethereum validity rollup; publish verified source, transaction receipts, raw datasets, and analysis scripts; compare CID representations; and inject provider loss. A second experiment will evaluate encrypted retrieval for different media sizes and geographic provider distributions. Further work should integrate verifiable credentials for inspectors, GS1-compatible identifiers, selective disclosure, sensor attestation, and formal verification of custody, quantity-conservation, initial-ization, and payment invariants.

References

- [1] T. Bosona, The role of blockchain technology in promoting traceability systems in agri-food supply chains, *Sensors* 23 (11) (2023) 5342. [doi:10.3390/s23115342](#).
- [2] M. Fiore, M. Mongiello, Blockchain technology to support agri-food supply chains: A comprehensive review, *IEEE Access* 11 (2023) 75311–75324. [doi:10.1109/ACCESS.2023.3296849](#).
- [3] G. Lv, C. Song, P. Xu, Z. Qi, H. Song, Y. Liu, Blockchain-based traceability for agricultural products: A systematic literature review, *Agriculture* 13 (9) (2023) 1757. [doi:10.3390/agriculture13091757](#).
- [4] T. V. Nguyen, H. C. Pham, M. N. Nguyen, L. Zhou, M. Akbari, Data-driven review of blockchain applications in supply chain management: Key research themes and future directions, *International Journal of Production Research* 61 (23) (2023) 8213–8235. [doi:10.1080/00207543.2023.2165190](#).
- [5] S. Charlebois, N. Latif, I. Ilahi, B. Sarker, J. Music, J. Vezeau, Digital traceability in agri-food supply chains: A comparative analysis of OECD member countries, *Foods* 13 (7) (2024) 1075. [doi:10.3390/foods13071075](#).
- [6] R. M. Ellahi, L. C. Wood, A. E.-D. A. Bekhit, Blockchain-driven food supply chains: A systematic review for unexplored opportunities, *Applied Sciences* 14 (19) (2024) 8944. [doi:10.3390/app14198944](#).
- [7] A. Pakseresht, S. A. Kaliji, K. Hakelius, Blockchain technology characteristics essential for the agri-food sector: A systematic review, *Food Control* 165 (2024) 110661. [doi:10.1016/j.foodcont.2024.110661](#).
- [8] R. M. Ellahi, L. C. Wood, A. E.-D. A. Bekhit, Blockchain-based frameworks for food traceability: A systematic review, *Foods* 12 (16) (2023) 3026. [doi:10.3390/foods12163026](#).
- [9] J. H. Khor, M. Sidorov, S. A. B. Zulqarnain, Scalable lightweight protocol for interoperable public blockchain-based supply chain ownership management, *Sensors* 23 (7) (2023) 3433. [doi:10.3390/s23073433](#).
- [10] L. A. Risso, G. M. D. Ganga, L. A. d. Santa-Eulalia, M. Godinho Filho, T. Chikhi, E. Mosconi, K. Zhang, A framework for modeling and simulating blockchain-based supply chain traceability systems, *International Journal of Production Economics* 278 (2024) 109408. [doi:10.1016/j.ijpe.2024.109408](#).
- [11] N. Sangeeta, S. Y. Nam, Blockchain and interplanetary file system (IPFS)-based data storage system for vehicular networks with keyword search capability, *Electronics* 12 (7) (2023) 1545. [doi:10.3390/electronics12071545](#).
- [12] S. K. Dwivedi, R. Amin, S. Vollala, Smart contract and IPFS-based trustworthy secure data storage and device authentication scheme in fog computing environment, *Peer-to-Peer Networking and Applications* 16 (1) (2023) 1–21. [doi:10.1007/s12083-022-01376-7](#).
- [13] S. Lamichhane, P. Herbe, Verifiable decentralized IPFS cluster: Unlocking trustworthy data permanency for off-chain storage, in: 2024 6th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), IEEE, 2024, pp. 1–4. [doi:10.1109/BRAINS63024.2024.10732266](#).
- [14] P. Kostamis, A. Sendros, P. S. Efraimidis, Data management in ethereum DApps: A cost and performance analysis, *Future Generation Computer Systems* 153 (2024) 193–205. [doi:10.1016/j.future.2023.11.026](#).
- [15] T. Lavaur, J. Detchart, J. Lacan, C. P. C. Chane, Modular zk-rollup on-demand, *Journal of Network and Computer Applications* 217 (2023) 103678. [doi:10.1016/j.jnca.2023.103678](#).
- [16] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis, B. Livshits, Analyzing and benchmarking ZK-rollups, in: 6th Conference on Advances in Financial Technologies (AFT 2024), Vol. 316 of Leibniz International Proceedings in Informatics, 2024, pp. 6:1–6:24. [doi:10.4230/LIPIcs.AFT.2024.6](#).
- [17] D. Grandjean, L. Heimbach, R. Wattenhofer, Ethereum proof-of-stake consensus layer: Participation and decentralization, in: *Financial Cryptography and Data Security: FC 2024 International Workshops*, Springer, 2024, pp. 253–280. [doi:10.1007/978-3-031-69231-4_17](#).
- [18] A. M. Ebrahimi, B. Adams, G. A. Oliva, A. E. Hassan, A large-scale exploratory study on the proxy pattern in ethereum, *Empirical Software Engineering* 29 (4) (2024) 81. [doi:10.1007/s10664-024-10485-1](#).
- [19] J. Li, Z. Zhao, Z. Su, W. Meng, Gas-expensive patterns detection to optimize smart contracts, *Applied Soft Computing* 145 (2023) 110542. [doi:10.1016/j.asoc.2023.110542](#).
- [20] Y. Liu, W. Song, FunRedis: Reordering function dispatch in smart contract to reduce invocation gas fees, in: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ACM, 2024. [doi:10.1145/3650212.3652146](#).
- [21] Raviprakash B, Rama Moorthy H, Daksha Malagatti, Arwin Sandesh Dsouza, Avinash N J, Praveen N, Optimizing arbitrage opportunities in automated market makers using scalable blockchain flash loans, in: 2025 International Conference on Artificial Intelligence and Data Engineering (AIDE), 2025, pp. 879–886. [doi:10.1109/AIDE64228.2025.10986867](#).